

Quant Research Platform

Full Documentation — Updated

0. Project Overview

A modular intraday futures research platform supporting 30+ instruments, built in Python and Streamlit. The application converts raw market data into enriched candle datasets, runs vectorized backtests, applies position sizing, and visualizes results — including a custom footprint chart and OHLCV chart built in HTML5 Canvas.

Designed for extensibility: adding a strategy, transform, or sizer requires only dropping a new file in the right folder. No registration, no imports to update.

1. Application Entry Point

The app is a single Streamlit process. Navigation is managed via `st.session_state.page`. The main file (`app.py`) routes between views:

```
home → data_formatter
      → backtester
      → analytics
      → footprint_chart
      → ohlcv_chart
      → monte_carlo
```

Each view exposes a single `render()` function. All navigation is done via `go_page(page: str)` helpers local to each view — no cross-view imports.

2. Folder Structure

```
project/
|-- data/
|   |-- raw_dbn/{type}/{asset}/{dataset}/ # .dbn.zst files
|   |-- parquet/{type}/{asset}/{dataset}/ # .parquet files
|   |-- trades/{name}.parquet           # backtest outputs
|-- transforms/
|-- strategies/
|-- position_sizing/
|-- monte_carlo/
|   |-- base.py
|   |-- bootstrap.py
|-- views/
|   |-- home.py
|   |-- data_formatter.py
|   |-- backtester.py
|   |-- analytics.py
|   |-- footprint_chart.py
|   |-- ohlcv_chart.py
|   |-- monte_carlo.py
```

Key structural rules:

- Raw DBN files are immutable. Never written to by the app.

- Parquet files are the working layer. Reprocessed by re-running a transform.
- The three-level folder hierarchy (type / asset / dataset) is enforced everywhere.
- Trades files are flat: data/trades/{name}.parquet. No sub-hierarchy.
- One Parquet file per calendar day, named YYYY-MM-DD.parquet.
- Asset folder names are always uppercase ticker symbols (e.g. ES, NQ, GC).

3. Asset Info & Tick Sizes

A shared ASSET_INFO dict is defined in each view file (backtester, analytics, monte_carlo, ohlcv_chart, footprint_chart). It maps asset ticker to tick size, and is used to automatically compute ticks_per_point and inject tick_size into strategy params — no manual input required.

The dict key is the asset folder name (always uppercase). ticks_per_point = 1 / tick_size is derived automatically. dollars_per_tick is looked up directly from the dict.

Asset	Tick Size	Ticks/Point	\$ / Tick	Category
ES	0.25	4	\$12.50	Equity Index
NQ	0.25	4	\$5.00	Equity Index
RTY	0.10	10	\$5.00	Equity Index
YM	1.00	1	\$5.00	Equity Index
MES	0.25	4	\$1.25	Equity Index (Micro)
MNQ	0.25	4	\$0.50	Equity Index (Micro)
M2K	0.10	10	\$0.50	Equity Index (Micro)
MYM	1.00	1	\$0.50	Equity Index (Micro)
ZN	0.015625	64	\$15.625	Rates
ZB	0.03125	32	\$31.25	Rates
ZF	0.0078125	128	\$7.8125	Rates
ZT	0.00390625	256	\$7.8125	Rates
SR3	0.0025	400	\$6.25	Rates
CL	0.01	100	\$10.00	Energy
QM	0.025	40	\$12.50	Energy
NG	0.001	1000	\$10.00	Energy
GC	0.10	10	\$10.00	Metals
MGC	0.10	10	\$1.00	Metals
SI	0.005	200	\$25.00	Metals
ZC	0.25	4	\$12.50	Grains
ZS	0.25	4	\$12.50	Grains
ZW	0.25	4	\$12.50	Grains
6E	0.00005	20000	\$6.25	FX
6J	0.0000005	2M	\$6.25	FX
6B	0.0001	10000	\$6.25	FX

Asset	Tick Size	Ticks/Point	\$/Tick	Category
6C	0.00005	20000	\$5.00	FX
BTC	5.00	0.2	\$25.00	Crypto

Note on non-equity assets: The IVB strategy hardcodes RTH as 09:30-16:00 NY. This is incorrect for CL (pit session 09:00-14:30), rates, FX, and metals. Results on non-equity assets should be treated with caution until per-asset session times are implemented.

4. Data Layer

4.1 Raw Data (Databento DBN)

Source files are .dbn.zst compressed archives from Databento. Each file covers one calendar day. The transform pipeline always consumes a pair of consecutive files — the previous evening's ETH session plus the current day's RTH session — to reconstruct a complete 23-hour trading session.

Schema used: **MBP-1** (top of book with trade data). Columns extracted:

Column	Rename	Description
side	side	B = buy aggressor, A = sell aggressor
price	price	Trade price
size	size	Trade size (contracts)
bid_px_00	bid_price_level	Best bid at time of trade
ask_px_00	ask_price_level	Best ask at time of trade
bid_sz_00	bid_size	Best bid size
ask_sz_00	ask_size	Best ask size
bid_ct_00	bid_count	Number of orders at best bid
ask_ct_00	ask_count	Number of orders at best ask
symbol	symbol	Contract month (e.g. ESU4)

4.2 Session Construction

- Load previous day and current day into DataFrames indexed by ts_event.
- Trim previous day to [22:00 UTC, >) and current day to [>, 21:00 UTC). Covers full Globex session.
- Concatenate and sort by timestamp.
- Convert index to America/New_York.
- Select the front-month contract by highest volume. Roll days are logged but still processed on the front month.
- Filter out prices below min_price (default 1000) to remove bad ticks.

4.3 Enriched 1-Minute Candle Schema

Produced by transforms/candles_1m.py. Each Parquet file stores 1-minute candles for one trading day covering 18:00 NY (previous evening) to 17:00 NY (current day) — the full Globex session. The index is a timezone-aware DatetimeIndex (America/New_York).

Column	Type	Description
open	float64	First trade price in bar
high	float64	Highest trade price
low	float64	Lowest trade price
close	float64	Last trade price
volume	int	Total contracts traded
buy_volume	int32	Contracts traded on buy aggressor side

Column	Type	Description
sell_volume	int32	Contracts traded on sell aggressor side
volume_delta	int32	buy_volume - sell_volume
volume_delta_pct	float	Delta as % of the smaller side (directional)
tick_volume	JSON str	{price: [buy_qty, sell_qty], ...} per price level
passive_orders	JSON str	{price: [size, order_count], ...} resting orders at best bid/ask

OHLC dtype change: open, high, low, close are now float64 (previously float32). Strategies, charts, and any code reading these columns must handle float64. No precision loss occurs — float64 is strictly more precise.

Gap filling: Zero-volume bars are forward-filled on OHLC from the previous close. Volume columns are filled with 0.

volume_delta_pct formula: When $\text{buy} > \text{sell}$, $\text{pct} = (\text{delta} / \text{sell}) * 100$. When $\text{sell} > \text{buy}$, $\text{pct} = (\text{delta} / \text{buy}) * 100$. When one side is zero, $\text{pct} = 100.0$.

tick_volume: Built by grouping all trades within the 1-minute window by price and side, then serialized to JSON. Limitation: multi-level fills are recorded only at the reported price.

passive_orders: Captures resting liquidity at the best bid/ask at the moment of each aggressor trade. Buy aggressor records ask levels above bar open (resistance). Sell aggressor records bid levels below bar open (support).

4.4 Indicators Transform (indicators_1m.py)

Reads enriched 1-minute candle Parquet files. Writes one indicators Parquet per day with 32 columns. Same filename as the source candle file. Follows the standard `run_all` transform interface.

VWAP Columns (28 columns)

Four anchored VWAPs, each with +/-1/2/3 sigma bands (7 columns each = 28 total):

Key	Anchor	Notes
vwap_bar_globex	18:00 NY (first bar)	Typical price = $(H+L+C)/3$ weighted by volume
vwap_bar_rth	09:30 NY	NaN before 09:30 and after 17:00
vwap_tick_globex	18:00 NY (first bar)	Uses tick_volume price distribution per bar
vwap_tick_rth	09:30 NY	NaN before 09:30 and after 17:00

Cumulative Delta (1 column)

`cumulative_delta = cumsum(buy_volume - sell_volume)`. Anchored at 18:00 NY. Resets every file. No cross-day accumulation.

Absorption Score (3 columns)

Computes the OFI-beta residual absorption signal. Valid only between 10:35 and 15:50 NY time (RTH only, after 60-bar warmup). All other bars are NaN.

Column	Description
beta	Rolling impact coefficient: $\text{cov}(\text{returns}, \text{delta}, 60) / \text{var}(\text{delta}, 60)$.
residual	Actual return minus expected return in ticks: $r_t - \text{beta}_t * \text{delta}_t$.
absorption_score	$-\text{sign}(\text{delta}) * \text{residual} / \text{rolling_std}(\text{residual}, 60)$. Score ≥ 2.0 is significant.

5. Views

5.1 Home (views/home.py)

Navigation hub only. Buttons routing to `data_formatter`, `backtester`, `analytics`, `footprint_chart`, `ohlcv_chart`, and `monte_carlo`. No state.

5.2 Data Formatter (views/data_formatter.py)

Purpose: Select a raw dataset and a transform script, run the transform, save Parquet output.

Flow: User selects input source (`raw_dbn` or `parquet`), navigates type > asset > dataset via cascading selectors, selects a transform, names the output folder. On Run: loads the transform module dynamically, calls `transform.run_all(input_folder, output_folder, skip_existing, on_progress)`. Progress bar and scrollable log container show output in real time. `skip_existing` flag skips dates already present in the output folder — enables fast incremental runs.

5.3 Backtester (views/backtester.py)

Purpose: Run a strategy on a candle dataset over a date range, inspect results, save trades.

ASSET_INFO integration

Defines `ASSET_INFO` and `HIDDEN_PARAMS = {"tick_size"}` at module level. When a run is executed, `tick_size` is automatically looked up from `ASSET_INFO` based on the selected asset folder name and injected into the strategy params dict before calling `strategy.run()`. The UI widget for `tick_size` is suppressed — `render_params()` skips any key in `HIDDEN_PARAMS`.

Ticks conversion

After the strategy returns trades, the backtester computes `ticks = pnl_points * ticks_per_point` where `ticks_per_point = ASSET_INFO[asset]["ticks_per_point"]`. This is stored in the in-memory trades DataFrame for display only — the saved trades file contains only `pnl_points`.

Chart view settings

Two modes: Candles before entry (N bars before entry_ts) or Fixed session time (start at e.g. 09:30). Always extends candles_after bars past the exit bar. Session files cover 18:00 NY the previous evening — the chart view filter strips pre-trade-date bars.

Trade notes

If the strategy populates a notes column (JSON string), the backtester renders a second row of metrics under the trade stats showing `breakout_time`, `retest_time`, `absorption_time`, `flip_count`, `POC`, `VAH`, `VAL`. Gracefully hidden for strategies that don't produce notes.

Save trades

Deduplicates by `DataFrame.equals()` before writing. Naming: `{dataset}_{strategy}_{start}_{end}.parquet`. Saved file contains only the strategy output columns — no ticks column.

5.4 Analytics (views/analytics.py)

Purpose: Load saved trade files, apply position sizing, compare multiple sized runs side by side.

ASSET_INFO integration

dollars_per_tick is derived automatically from the trades filename. The first token of the filename (before the first underscore) is treated as the asset ticker and looked up in ASSET_INFO. Example:
ES_ivb_model2_v2_2024-01-01_2024-12-31.parquet → asset = ES → dollars_per_tick = 12.50. This value is injected into sizer params automatically — no manual input needed.

Instance builder

User creates N instances. Each instance independently specifies a trades file, a sizer, and a label. Shared keys (account_size, dollars_per_tick) injected automatically. account_size is set once at the top of the page. dollars_per_tick comes from ASSET_INFO.

Metrics (dollar-denominated)

Total P&L, Final Equity, Max Drawdown (\$), Max Drawdown (%), Sharpe (annualized, daily grain, ddof=1, sqrt(252)), Win Rate, Profit Factor, Skipped Trades.

5.5 Footprint Chart (views/footprint_chart.py)

Purpose: Interactive candlestick/footprint chart with order flow overlays, built in HTML5 Canvas. Fully static after render — no Python callbacks. tick_size is read from TICK_SIZES dict for correct price snapping on H-lines.

Data selectors

- Candles — cascading type > asset > dataset selector, end date, days to show (1-5).
- Big trades (optional) — separate cascading selector, ETH/RTH size thresholds.
- Indicators (optional) — separate cascading selector, reads indicators parquet.

Feature	Description
Candlesticks	Standard OHLC with wicks
Delta panel	Bar-by-bar volume delta above chart, color-coded green/red
Footprint	Buy/sell volume per price tick overlaid on each candle
Passive orders	Resting bid/ask liquidity, color-coded by position relative to open. Highlight threshold configurable.
Big trades	Circles sized proportional to trade size. Blue = buy, purple = sell. Separate ETH/RTH thresholds.
VWAP	Selectable: bar globex, bar RTH, tick globex, tick RTH. Independent std band toggles (1s, 2s, 3s).
CVD panel	Cumulative volume delta sub-panel (togglable, 20% of chart height)
Absorption overlay	Candle highlight where absorption_score >= threshold. Orange = buy absorbed, purple = sell absorbed.
Volume Profile	Peak-based algorithm (see Section 8). Click-drag to select range. Computes POC, VAH, VAL (70% VA). Delta bars
H/V/Diagonal lines	Manual reference lines. Edit mode allows drag-to-move. Delete mode removes on click.

Navigation: scroll = pan horizontal, ctrl+scroll = zoom X, shift+scroll = zoom Y, drag = pan both axes.

5.6 OHLCV Chart (views/ohlcv_chart.py)

Purpose: Clean multi-timeframe candlestick chart for 30+ instruments. No footprint, no order flow. Designed for fast browsing across assets and timeframes.

Features

- Multi-timeframe resampling from 1m source: 1m, 5m, 15m, 30m, 1h, 4h, 1D, 1W, 1M.
- Volume sub-panel (buy/sell color-coded if available).
- Drawing tools: H-line (price-snapped to tick_size), V-line, Diagonal, Delete, Clear.
- Dual-chart mode — two independent charts side by side with optional drawing sync.
- Sync toggle shares H/V/diagonal lines between both charts when enabled.
- tick_size read from TICK_SIZES for correct price snapping per asset.
- Tooltip shows OHLCV, buy/sell volume, timestamps in NY time.

Timeframe buttons

Each timeframe (1m through 1M) is a button. Active timeframe stored in `st.session_state[f"{prefix}_tf"]`. Changing timeframe triggers `st.rerun()`.

5.7 Monte Carlo (`views/monte_carlo.py`)

Purpose: Simulate thousands of equity paths by resampling the trade history. Quantifies uncertainty in strategy performance and stress-tests position sizing.

ASSET_INFO integration

`dollars_per_tick` derived from trade filename (same as Analytics). Injected into `sizer_params` at run time — not shown in UI.

Page layout

Section 1 — Trades: dropdown of saved files from `data/trades/`. Section 2 — Position Sizing: sizer selection, account size, sizer-specific params. Section 3 — Simulation: MC method, ruin definition, simulation params. Section 4 — Results: fan chart + metrics table.

Fan chart bands (outermost first)

- 3s band (0.15th–99.85th percentile) — faintest fill
- 2s band (2.5th–97.5th percentile) — medium fill
- 1s band (16th–84th percentile) — brightest fill
- Sampled paths: 10% of all paths as faded grey lines
- Featured paths: best final, best peak, worst final, lowest trough
- Median path: thick white line on top

6. Plugin Systems

6.1 Transform Scripts

A transform is a Python file in `transforms/`. Loaded dynamically by the Data Formatter. The stem of the filename becomes its display name in the UI.

```
def run_all(
    input_folder: str,          # path to folder of input files
    output_folder: str,         # path to output Parquet folder
    skip_existing: bool,        # skip dates already present in output
    on_progress: callable,      # on_progress(current, total, message)
) -> None: ...
```

6.2 Strategy Scripts

A strategy is a Python file in `strategies/`. Loaded dynamically by the Backtester. `__init__.py` and `base.py` are excluded.

```
PARAMS = { 'param_name': default_value, ... } # optional

def run(
    folder_path: Path,
    start_date: pd.Timestamp,
    end_date: pd.Timestamp,
    params: dict,
) -> pd.DataFrame: ...
```

Required output columns:

```
date, direction, entry_time, exit_time,
entry_price, exit_price, sl, tp, exit_reason, pnl_points
```

Optional:

```
notes # JSON string with strategy-specific metadata
```

pnl_points = `exit_price - entry_price` for longs, `entry_price - exit_price` for shorts. Return empty DataFrame with correct columns when no trades generated.

tick_size is automatically injected into params by the backtester from `ASSET_INFO` — keep it in `PARAMS` as a default but mark it in `HIDDEN_PARAMS` to suppress the UI widget.

No ticks column in output. The backtester computes ticks from `pnl_points` after the strategy returns. Strategies must not compute or store ticks themselves.

6.3 Position Sizer Scripts

A sizer is a Python file in `position_sizing/`. Loaded dynamically by Analytics and Monte Carlo.

```
PARAMS = {
    'account_size': 100_000.0, # injected from shared defaults
    'dollars_per_tick': 12.50, # injected from ASSET_INFO
    'other_param': default,
}

def apply(trades: pd.DataFrame, params: dict) -> pd.DataFrame: ...
```

`apply()` must return a copy with additional columns: `size` (contracts), `trade_pnl` (`ticks * dollars_per_tick * size`), `equity` (`account_size + trade_pnl.cumsum()`). Skipped trades (`size` rounds to zero) excluded from output, count stored in `result.attrs['skipped_trades']`

6.4 Monte Carlo Scripts

A Monte Carlo simulation is a Python file in `monte_carlo/`. `base.py` is excluded.

```
PARAMS = { 'n_paths': 1000, 'seed': 42, ... }
```

```
def run(trades, sizer_module, sizer_params, params) -> dict: ...
```

Must return a dict with at minimum: `equity_matrix` (`np.ndarray` shape `(n_paths, n_trades+1)`), `n_trades` (`int`), `method` (`str`).

7. Volume Profile Algorithm (Peak-Based)

Used in both the Footprint Chart and the IVB strategy. Replaces the old simple max-volume POC algorithm. Produces tighter, more meaningful value areas on bimodal and multimodal profiles.

Step 1 — Build raw levels: Aggregate tick_volume JSON across all bars in the selected range into a dict {price: total_volume}.

Step 2 — Smooth: Apply a 3-tick rolling average to volumes. For each index i , average volumes $[i-1..i+1]$ (clamped at edges). Removes single-tick spikes without changing distribution shape.

Step 3 — Find local maxima: A peak is where smoothed $[i] > \text{smoothed}[i-1]$ AND smoothed $[i] > \text{smoothed}[i+1]$. This is equivalent to finding where the discrete derivative flips from positive to negative. Endpoints checked separately.

Step 4 — Cluster peaks: Peaks within 4 index positions (4 ticks) of each other are merged into one cluster. The highest smoothed value per cluster survives. Top 5 clusters taken by smoothed volume.

Step 5 — Refine to raw peak: For each cluster peak index, scan ± 3 indices in the raw (unsmoothed) volumes array. The index with the highest raw volume becomes the POC candidate price. Ensures the final POC is the actual highest-volume tick, not the smoothed one.

Step 6 — Expand value area: For each POC candidate, expand outward (always adding the side with higher adjacent volume) until cumulative volume $\geq 70\%$ of total. Record VAH and VAL.

Step 7 — Pick tightest: Compare VAH - VAL across all candidates. The candidate with the smallest range wins.

Step 8 — Refine POC within VA: After selecting the winning VA, scan all price levels between VAL and VAH in the raw profile. The level with the highest raw volume becomes the final POC. This corrects cases where the smoothed peak and the true highest-volume tick differ.

Key insight: On bimodal profiles, the old algorithm (expand from global max) often produced a VA spanning the valley between two peaks. The peak-based algorithm finds the densest cluster and produces the tightest meaningful VA instead. Both the footprint chart and IVB strategy use identical logic.

8. Shared Patterns

8.1 Cascading Folder Selector

Used identically in Backtester, Analytics, Footprint Chart, OHLCV Chart, and Monte Carlo. Returns (folder_path, asset_type, asset, dataset). Widget keys encode all parent selections to force reset when parent changes.

8.2 Dynamic Module Loading

Transforms, strategies, sizers, and Monte Carlo scripts all loaded the same way:

```
spec = importlib.util.spec_from_file_location(name, path)
module = importlib.util.module_from_spec(spec)
spec.loader.exec_module(module)
```

No `__init__.py` needed. No import registration. Modules re-loaded fresh on each button click.

8.3 Session State Persistence

Backtester, Analytics, and Monte Carlo store run results in `st.session_state` so results survive widget interaction. Always initialize keys at the top of `render()` to avoid `KeyError` on first load. Never use the same key for both a widget and a session state assignment.

8.4 HIDDEN_PARAMS

Defined at module level in `backtester.py` as `HIDDEN_PARAMS = {"tick_size"}`. Keys in this set are skipped by `render_params()` and never shown as UI widgets. Instead, `execute_run()` injects the correct value from `ASSET_INFO` before calling the strategy. This pattern can be extended for any param that should be auto-derived rather than user-entered.

9. Data Flow Summary

```
raw_dbn/{type}/{asset}/{dataset}/  
  ↓ (Data Formatter + candles_1m.py transform)  
parquet/{type}/{asset}/{dataset}/YYYY-MM-DD.parquet [advanced candles – float64 OHLC]  
  ↓ (Data Formatter + indicators_1m.py transform)  
parquet/{type}/{asset}/{dataset}/YYYY-MM-DD.parquet [indicators]  
  ↓ (Backtester + strategy)  
trades/{name}.parquet [pnl_points only – no ticks column]  
  ↓ (Analytics + position sizer)  
sized equity curve + dollar metrics  
  ↓ (Monte Carlo + position sizer)  
fan chart + statistical metrics
```

Footprint Chart and OHLCV Chart read directly from `parquet/` and render without output.

10. Monte Carlo Detail

10.1 Bootstrap Monte Carlo (bootstrap.py)

Resamples trades WITH replacement. Each path draws `n_sample` trades from the original trade set, allowing the same trade to appear multiple times and some trades not at all. Models uncertainty in the edge estimate rather than just testing sequence sensitivity.

Parameter	Default	Description
<code>n_paths</code>	1000	Number of simulated equity paths
<code>n_trades</code>	0	Trades per path. 0 = match file length
<code>seed</code>	42	Random seed for reproducibility

10.2 Metrics Table

Metric	Description
Median final equity	50% of paths ended above/below this. The honest number.
Mean final equity	Pulled up by outlier paths (especially with Kelly sizing).
Final equity 1s/2s/3s	Final equity range at each Gaussian percentile pair, in dollars and %.
Worst drop below start (3s)	Deepest drop below starting capital at the 3-sigma tail.
Median drop below start	Typical bad path's deepest drop below starting capital.
% paths ever below start	Robustness metric. Low = strategy rarely threatens capital.
Median Sharpe	Annualized Sharpe (daily grain, <code>ddof=1</code> , <code>sqrt(252)</code>) at median path.
P(ruin)	Fraction of paths that hit the selected ruin floor. Shown only if threshold selected.

10.3 base.py Utilities

`build_equity_curve(trades, sizer_module, sizer_params)`: applies sizer to a trades DataFrame and returns equity curve as numpy array starting from `account_size`. Shape: `(n_trades + 1,)`.

`run_paths(trades, sizer_module, sizer_params, resample_fn, n_paths, seed)`: runs `n_paths` simulations using a provided `resample_fn(trades, rng, path_index) -> pd.DataFrame`. Returns `equity_matrix` of shape `(n_paths, n_trades+1)`.

11. Known Limitations and Design Notes

- **OHLC dtype**: open/high/low/close are float64. Previously float32. Any code doing dtype-specific operations must be updated.
- **Session times**: IVB strategy hardcodes RTH as 09:30–16:00 NY. Wrong for CL, rates, FX, metals. Per-asset session time config is on the roadmap.
- **Tick volume accuracy**: Multi-level fills recorded only at the reported price. Single large orders crossing price levels are not decomposed.
- **Sharpe annualization**: Uses 252 trading days. Correct for RTH-only intraday strategies.

- **Direction convention:** long and short must be lowercase everywhere. Both Backtester and Analytics depend on exact string matching.
- **Contract rolls:** Transform selects front month by volume. Roll days flagged in log but processed on front month. Cross-roll backtests have a one-day discontinuity — no Panama or ratio adjustment applied.
- **Passive orders:** Takes first occurrence of each price level per aggressor side per bar, not aggregate. Does not track order book updates within a bar.
- **Trade deduplication:** DataFrame.equals() check including column order and index. Two runs with identical trades but different index types will not be detected as duplicates.
- **VWAP RTH window:** Valid 09:30–17:00 NY. Bars outside this window are NaN.
- **Absorption score warmup:** Requires 60 RTH bars for beta window + 60 bars for sigma_u window. Signal valid from approximately 11:35 NY time. Not suitable for early-session strategies.
- **Monte Carlo bootstrap:** Assumes trade independence. Autocorrelated strategies may produce overoptimistic confidence intervals.
- **Non-equity asset testing:** OHLCV data for 30 assets available for 1 year. Enriched tick_volume/passive_orders data (required by IVB strategy) only available for ES/NQ from Databento subscription period.

12. Development Roadmap

Item	Status
Fix pnl_points/ticks — strategy outputs pnl_points only, backtester converts	DONE
ASSET_INFO + HIDDEN_PARAMS — auto tick_size injection, no manual widget	DONE
Volume profile peak-based algorithm	DONE
IVB Model 2 v2 — absorption candle entry + direction flips + notes column	DONE
OHLCV Chart — multi-timeframe, dual chart, drawing tools	DONE
Backtester chart view settings — candles before entry / fixed session time	DONE
Data transform speed improvements	DONE
Fix session times per asset (09:30 hardcoded — wrong for non-equity)	TODO
Passive order entry type (large resting bid/ask as entry signal)	TODO
Consecutive absorption candles signal	TODO
Earlier entries — reduce lag from absorption to entry	TODO
Minimum risk threshold — extend TP when SL too close	TODO
Trail SL to absorption level when in profit	TODO
Walk-forward validation (needs multi-year data)	TODO
Prop firm Monte Carlo — daily loss / max drawdown constraints	TODO
IVB Model 2 — full order flow confirmation version	TODO